

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Deepa preya H	Reg.No.:	192421047
Department:	Information technology	Date:	30.08.26

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

*I **Deepa preya H / 192421047** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. DMA-Based Data Transfer from I/O Device to Memory

Algorithm

1. Initialize the DMA controller.
2. Specify the I/O device, memory starting address, and data size.
3. Enable DMA transfer.
4. CPU continues other tasks while DMA transfers data.
5. DMA requests the system bus.
6. CPU grants the bus to DMA.
7. DMA transfers data directly from the I/O device to memory.
8. After completion, DMA releases the bus and generates an interrupt.
9. CPU handles the interrupt and verifies the transfer.

Pseudocode

ALGORITHM DMA_Data_Transfer

BEGIN

 Initialize DMA_Controller

 Set DMA.Source $\leftarrow$ IO_Device

 Set DMA.Destination $\leftarrow$ Memory_Address

 Set DMA.Transfer_Size $\leftarrow$ Data_Size

 Set DMA.Mode $\leftarrow$ IO_TO_MEMORY

 Enable DMA

 WHILE DMA.Transfer_Complete = FALSE DO

 CPU.Execute_Other_Tasks()

 IF DMA.Bus_Request = TRUE THEN

 CPU.Grant_Bus_To_DMA()

 END IF

 END WHILE

 DMA.Release_Bus()

 DMA.Generate_Interrupt()

 IF Interrupt_Received = TRUE THEN

 CPU.Handle_Interrupt()

 Verify_Transfer()

 IF Transfer_Successful = TRUE THEN

 Display "DMA Transfer Completed Successfully"

```
ELSE
    Display "DMA Transfer Failed"
END IF
END IF
```

```
END
```

2. Vectored Interrupt Handling for Multiple Devices

Algorithm

1. Initialize the interrupt controller.
2. Assign a priority and interrupt vector to each device.
3. Continuously monitor interrupt requests.
4. Identify the highest-priority pending interrupt.
5. Obtain its interrupt vector.
6. Use the vector to locate the appropriate service routine.
7. Execute the service routine.
8. Clear the interrupt request.
9. Restore CPU state and continue normal execution.

Pseudocode

ALGORITHM Vectored_Interrupt_Handler

BEGIN

Initialize Interrupt_Controller

Assign Priority and Vector to Each Device

WHILE System_Running = TRUE DO

IF Interrupt_Request_Exists() = TRUE THEN

Device ← Get_Highest_Priority_Device()

Vector ← Get_Interrupt_Vector(Device)

Save_CPU_State()

Service_Routine ← Interrupt_Vector_Table[Vector]

CALL Service_Routine(Device)

Clear_Interrupt(Device)

Restore_CPU_State()

```
        Resume_Normal_Execution()

    END IF

END WHILE

END
```

Service Routine

PROCEDURE Service_Routine(Device)

```
BEGIN

    IF Device = Keyboard THEN
        Read_Keyboard_Data()

    ELSE IF Device = Network THEN
        Process_Network_Data()

    ELSE IF Device = Disk THEN
        Process_Disk_Request()

    ELSE
        Handle_Unknown_Interrupt()

    END IF

END
```

3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

The selection is based on the **latency and bandwidth requirements** specified in the question.

Algorithm

1. Read the device's latency and bandwidth requirements.
2. Define suitable thresholds.
3. If low latency and high bandwidth are required, select memory-mapped I/O.
4. Otherwise, select I/O-mapped I/O.
5. Configure the selected I/O method.
6. Perform the data transfer.
7. Monitor performance.
8. Change the I/O method if requirements change.

Pseudocode

ALGORITHM Dynamic_IO_Selection

```
BEGIN
```

```

Read Latency_Requirement
Read Bandwidth_Requirement

Set Latency_Threshold ← LOW
Set Bandwidth_Threshold ← HIGH

IF Latency_Requirement <= Latency_Threshold AND
   Bandwidth_Requirement >= Bandwidth_Threshold THEN

    IO_Mode ← MEMORY_MAPPED_IO

ELSE

    IO_Mode ← IO_MAPPED_IO

END IF

Configure_IO_Mode(IO_Mode)

IF IO_Mode = MEMORY_MAPPED_IO THEN
    Perform_Memory_Mapped_Transfer()
ELSE
    Perform_IO_Mapped_Transfer()
END IF

Monitor_IO_Performance()

IF Requirements_Changed = TRUE THEN
    REPEAT Dynamic_IO_Selection
END IF

END

```

4. Bus Arbitration for PCIe, USB and Thunderbolt

The objective is to manage **multiple devices requesting communication resources** over PCIe, USB, and Thunderbolt interfaces. The question specifically requires these three interfaces.

Algorithm

1. Initialize the bus arbitration system.
2. Monitor requests from PCIe, USB, and Thunderbolt devices.
3. Collect all active requests.
4. Assign priority to each request.
5. Select the highest-priority request.
6. Grant access to the selected device/interface.
7. Allow data transfer.

8. Release the bus/resource after transfer.
9. Check for pending requests.
10. Repeat until communication is complete.

Pseudocode

ALGORITHM Bus_Arbitration

BEGIN

Initialize PCIe

Initialize USB

Initialize Thunderbolt

WHILE System_Running = TRUE DO

Requests $\leftarrow$ Collect_Bus_Requests()

IF Requests $\neq$ EMPTY THEN

FOR EACH Request IN Requests DO

Assign_Priority(Request)

END FOR

Selected_Request $\leftarrow$ Select_Highest_Priority(Requests)

IF Selected_Request.Interface = PCIe THEN

Grant_PcIe_Access(Selected_Request)

ELSE IF Selected_Request.Interface = USB THEN

Grant_USB_Access(Selected_Request)

ELSE IF Selected_Request.Interface = Thunderbolt THEN

Grant_Thunderbolt_Access(Selected_Request)

END IF

Perform_Data_Transfer(Selected_Request)

Release_Bus_Resource(Selected_Request)

Update_Request_Status(Selected_Request)

ELSE

Continue_Normal_Operation()

END IF

END WHILE

END

5. Hybrid I/O Communication Mechanism

The question requires **polling for low-speed devices** and **DMA with interrupts for high-speed devices**.

Algorithm

1. Identify the I/O device.
2. Determine its speed.
3. If the device is low-speed, use polling.
4. If the device is high-speed, configure DMA.
5. Start DMA transfer.
6. CPU performs other tasks while DMA operates.
7. DMA generates an interrupt after completion.
8. CPU handles the interrupt.
9. Verify the transfer.
10. Continue normal operation.

Pseudocode

ALGORITHM Hybrid_IO_Communication

BEGIN

Initialize_IO_System()

WHILE System_Running = TRUE DO

Device ← Detect_IO_Device()

Speed ← Determine_Device_Speed(Device)

IF Speed = LOW_SPEED THEN

WHILE Data_Available(Device) = FALSE DO

Poll(Device)

END WHILE

Data ← Read_Data(Device)

Process_Data(Data)

ELSE IF Speed = HIGH_SPEED THEN

Initialize_DMA_Controller()


```
Set DMA.Source ← Device
Set DMA.Destination ← Memory
Set DMA.Transfer_Size ← Required_Data_Size

Enable_DMA_Interrupt()

Start_DMA_Transfer()

CPU.Execute_Other_Tasks()

WAIT FOR DMA_Interrupt

IF DMA_Interrupt_Received = TRUE THEN

    Handle_DMA_Interrupt()

    IF Transfer_Successful = TRUE THEN
        Process_Transferred_Data()
    ELSE
        Handle_Transfer_Error()
    END IF

END IF

END IF

END WHILE

END
```